



Analytics & Big Data

Session 9: Big data 2

Prof. Dr. Gerit Wagner

(2026-05-04)

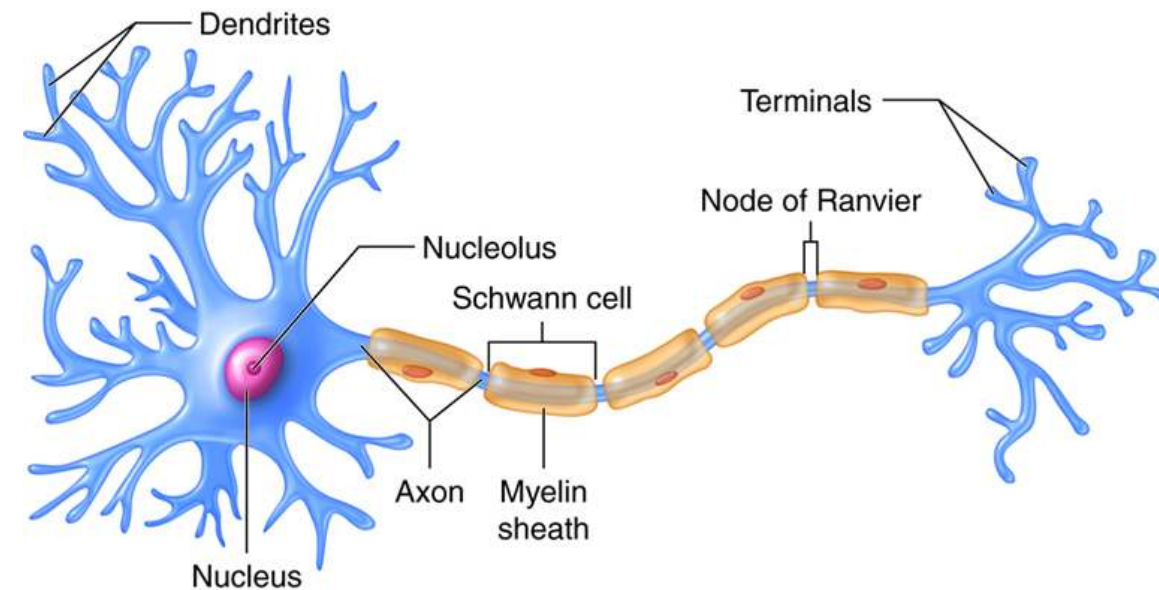
Learning objectives

- **Explain and interpret** the core ideas and mathematical operations underlying artificial neural networks
- **Distinguish** between major neural network architectures (MLP, CNN, RNN, TM) and identify the types of data for which they are most appropriate.
- **Understand** the design and training process of neural networks, including architecture selection and optimization.



Foundations

From biological neurons to artificial neural networks



Key milestones

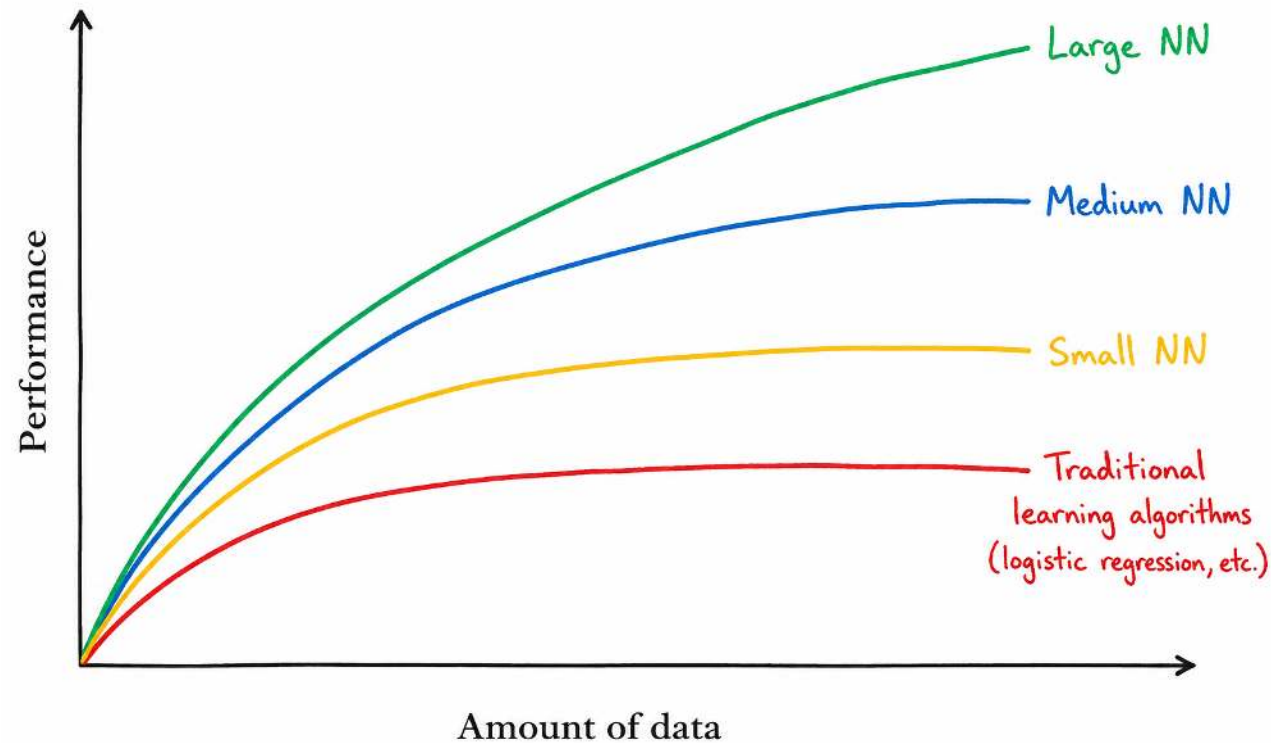
- **1943** (Warren McCulloch & Walter Pitts) — Mathematical model of artificial neurons
- **1958** (Frank Rosenblatt) — Development of the *Perceptron*
- **2010s** (Geoffrey Hinton, Yann LeCun & Yoshua Bengio) — Revival of modern deep learning
- **2020s** (GPT models, OpenAI) — Emergence and widespread adoption of large language models (LLMs) and multimodal AI systems

Performance of machine learning and neural networks



Key drivers of recent progress in neural networks

- Availability of large-scale datasets
- Increasing computational power (especially GPUs/TPUs)
- Advances in algorithms and neural network architectures
- Specialization of neural networks:
 - Convolutional Neural Networks (CNNs) for image and video recognition
 - Recurrent Neural Networks (RNNs) for sequential and time-series data
 - Transformers for language processing and generative AI
 - Graph Neural Networks (GNNs) for network and relational data
 - Diffusion Models for image and media generation



In small-data regimes, experiments are needed to identify algorithms that perform best for a given dataset.

The perceptron



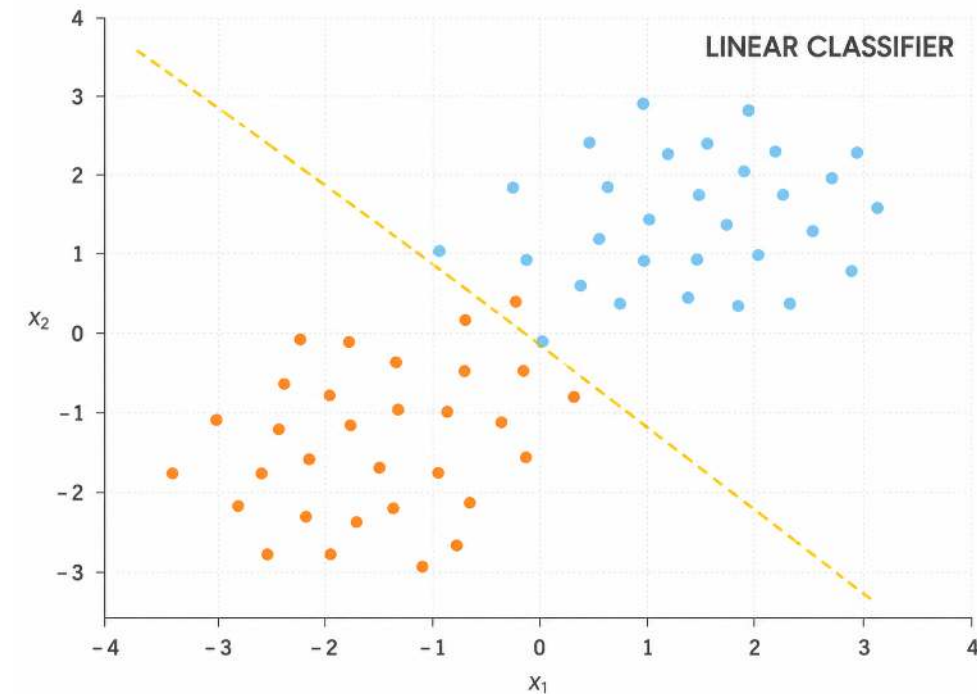
A perceptron is one of the earliest forms of neural networks. A **classical perceptron** is characterized by:

$$z = w^T x + b$$

followed by a **threshold activation**:

$$\hat{y} = \begin{cases} 1 & \text{if } w^T x + b > 0 \\ 0 & \text{otherwise} \end{cases}$$

- This corresponds to a linear classifier
- The training of the perceptron consists of feeding it multiple training samples and calculating the output for each of them
- After each sample, the weights w are adjusted to minimize a loss function, which quantifies the difference between the predicted and true outputs



From perceptron to deep neural networks



Limitation of a single perceptron: It cannot build abstract intermediate representations and can only learn linear decision boundaries.

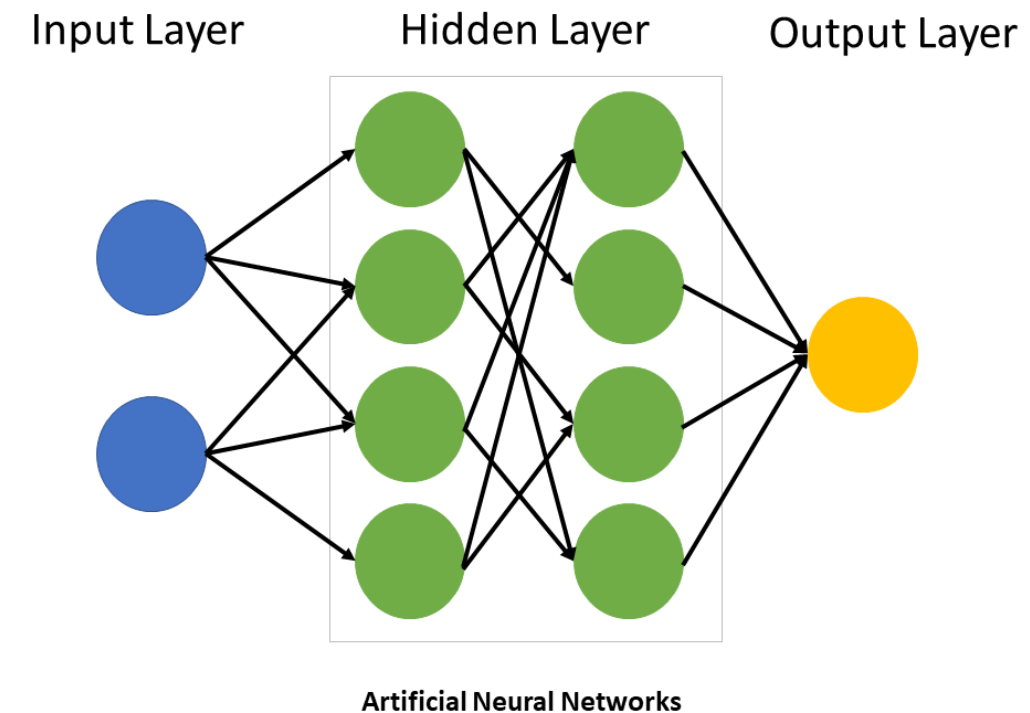
Solution: Multilayer neural networks combine:

- **Multiple hidden layers**, which enable increasingly abstract representations
- **Activation functions**, which make nonlinear learning possible

Layers:

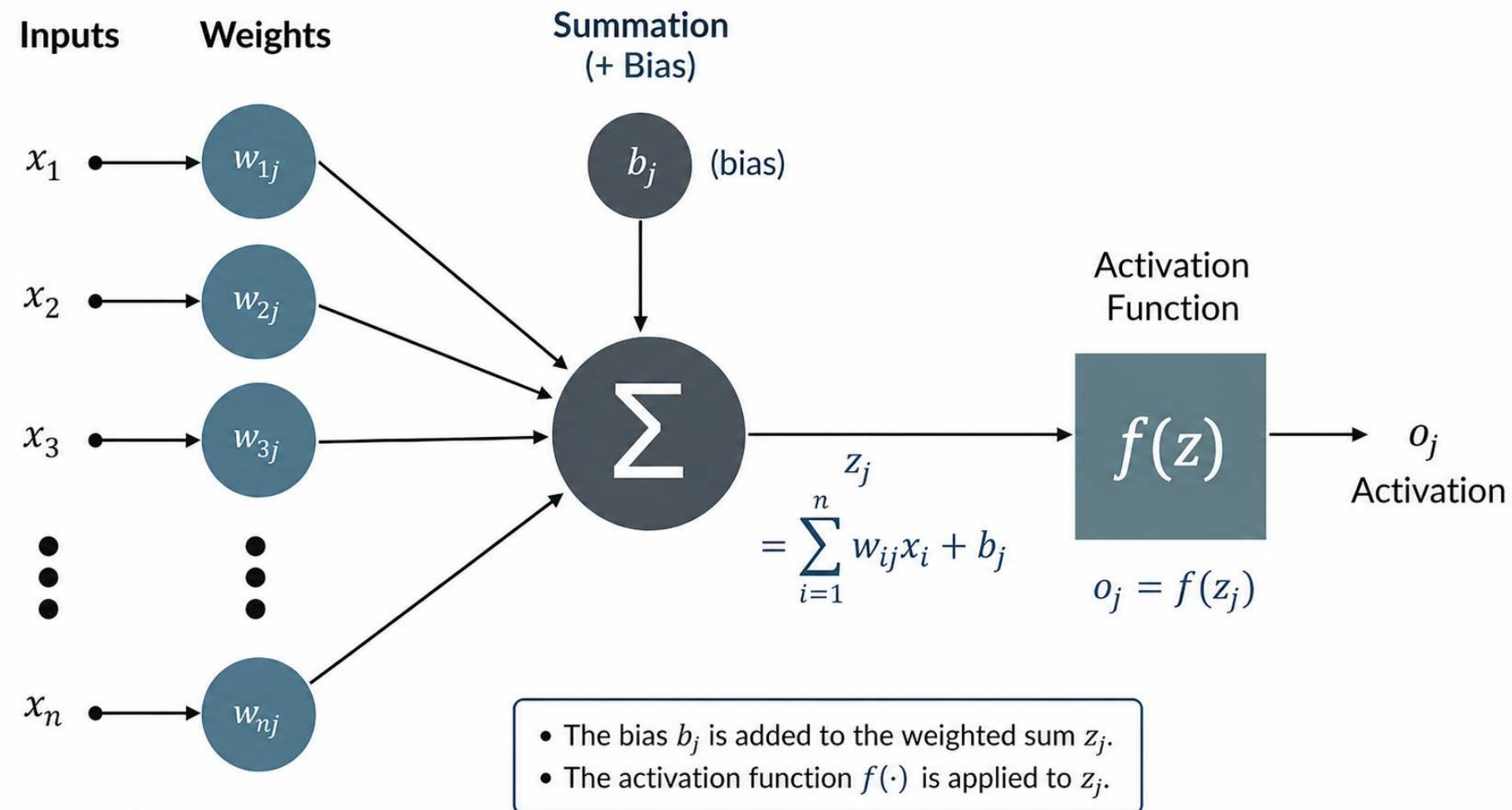
- Input neurons: raw data (e.g., images, audio, video)
- Hidden neurons: the weights and biases store abstract representations learned from the data (e.g., edges, parts of an image)
- Output neuron: prediction (e.g., one neuron for binary, multiple for multi-class)

Note: the number of layers and neurons is specified ex-ante and does not change in the training process.



 Grant Anderson (3Blue1Brown) provides a series of instructive animated explanations, including one on [Neural Networks](#).

Functionality of a neuron



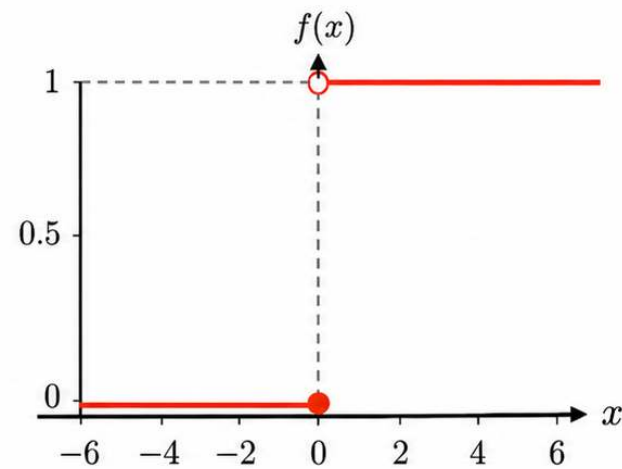
In a classical perceptron, the activation is typically binary (0 or 1), representing whether the neuron “fires.”
In modern neural networks, activations are usually continuous-valued scalars determined by the activation function.

Types of activation functions



Step

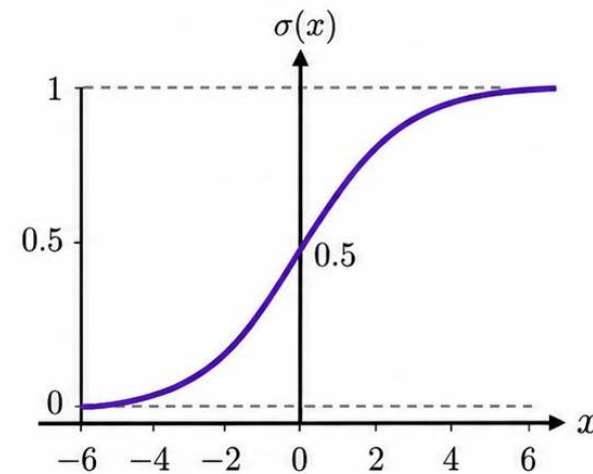
$$f(x) = \begin{cases} 0, & x \leq 0 \\ 1, & x > 0 \end{cases}$$



Note: Perceptron

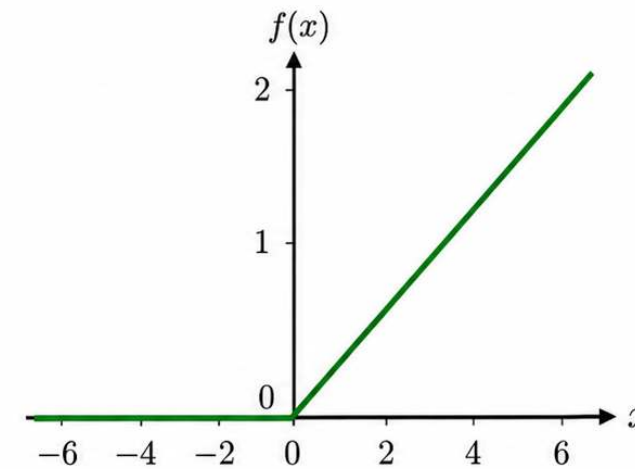
Sigmoid

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$



ReLU

$$f(x) = \max(0, x) = \begin{cases} 0, & x \leq 0 \\ x, & x > 0 \end{cases}$$



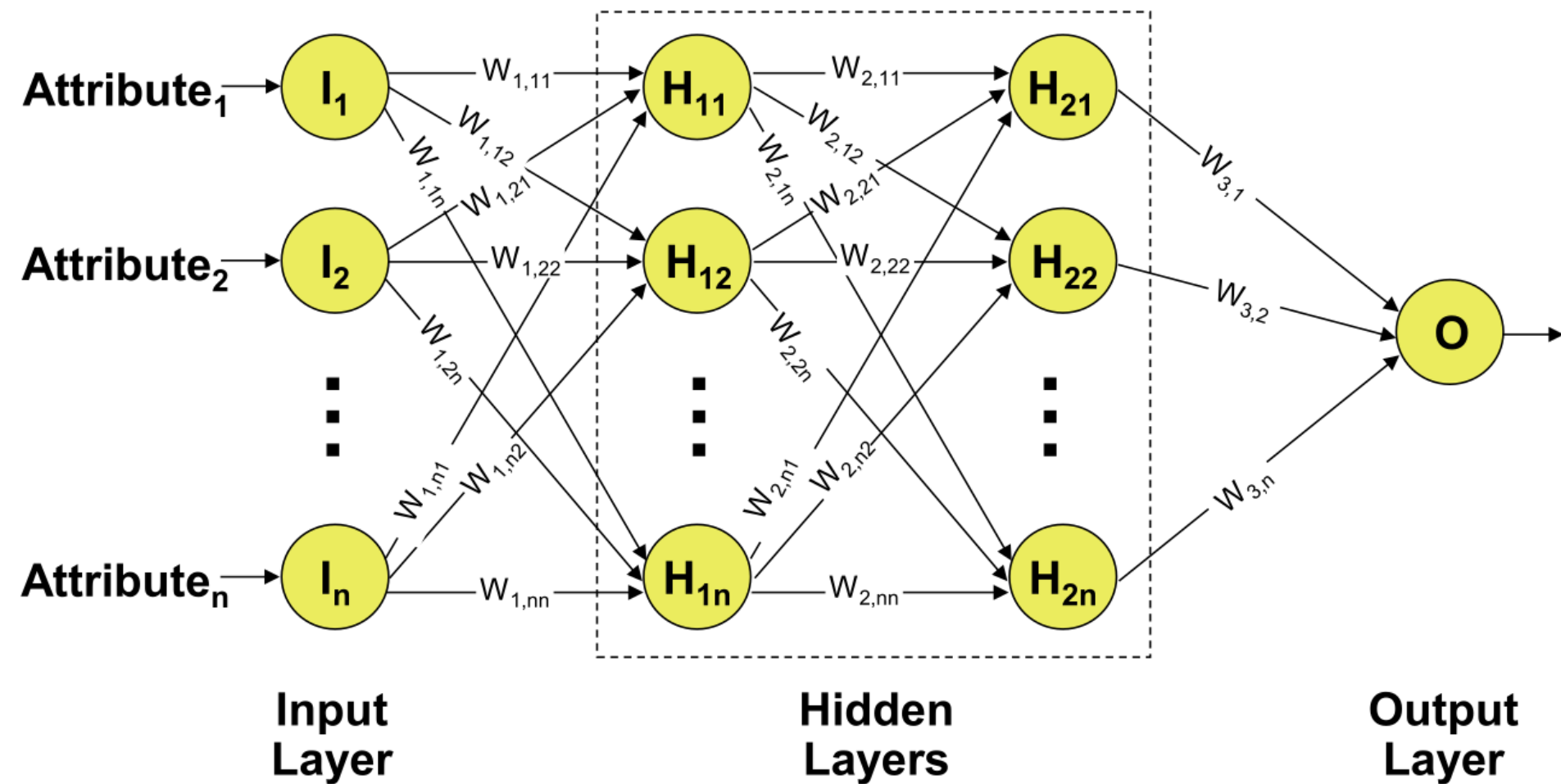
Note: Efficient computation

ReLU: Rectified Linear Unit.

Training of a multilayer perceptron



- Weights are initialized with carefully selected random values
- For each training item, the predicted output is calculated (“forward pass”)



Loss function



The network's prediction error is measured with a **loss function**.

For one training example i :

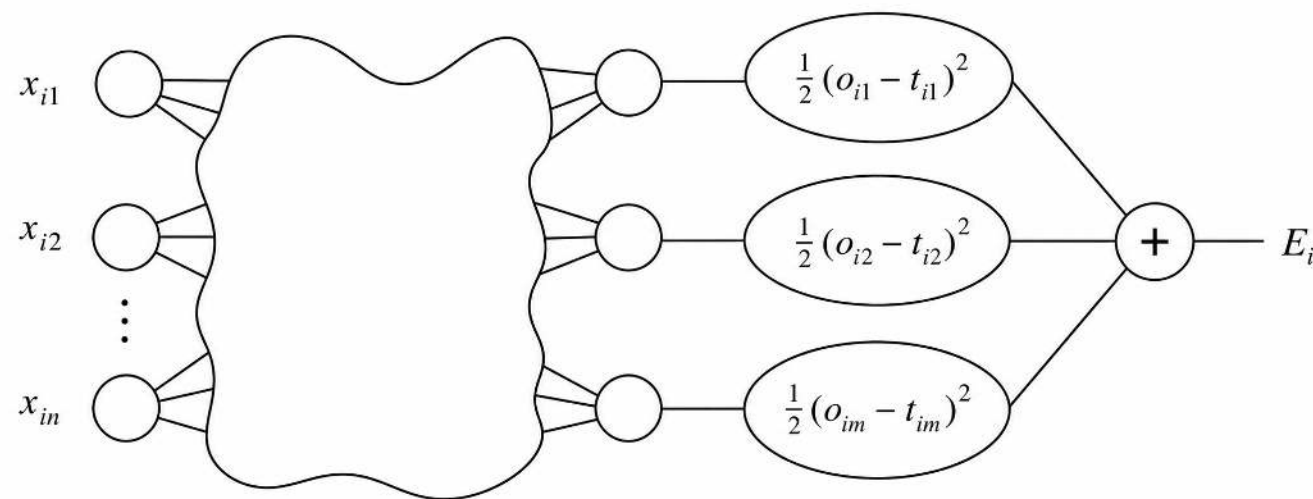
$$E_i = \frac{1}{2} \sum_{j=1}^m (o_{ij} - t_{ij})^2$$

This is the squared error loss across all **output neurons** j .

Across all h training examples, the total loss is:

$$E = \sum_{i=1}^h E_i$$

Backpropagation adjusts the weights to reduce this total loss.



Backpropagation



Backpropagation calculates the gradients of the loss function with respect to the network's weights. These gradients are then used by an optimizer, to iteratively update the weights and reduce the difference between predicted and true outputs.

```
1 initialize weights and biases
2
3 repeat until stopping criterion is met:
4
5     total_error = 0
6
7     for each training example:
8
9         pass input forward through all layers
10        compare predicted output with target value
11        add error to total_error
12        propagate error backward through all layers
13        update weights and biases
14
15    check whether total_error is small enough
```

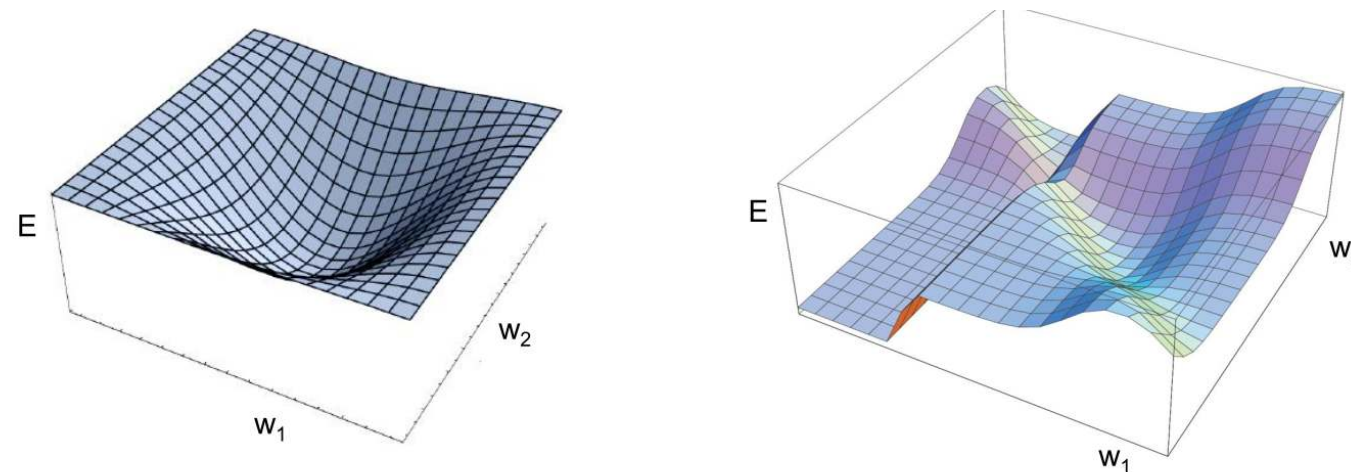
Using the loss function to adjust weights

The loss function E has to be minimized. Because it depends on the output neurons o_j , it automatically depends on their weights to the precedent layer(s):

$$o_j = f(s(x)_j) \text{ with } s(x)_j = \sum_k^n w_{jk} \cdot x_k$$

Thus, the weights have to be found where E is minimal.

Examples of the loss function with **two weights** (simplified):



Gradient descent

To minimize the loss function E the backpropagation algorithm uses the method of gradient descent. This method searches those weights, where the vector containing the partial first derivatives of the loss function ∇E (gradient) is equal to the zero vector (minimum):

$$\nabla E = \left(\frac{\partial E}{\partial w_1}, \frac{\partial E}{\partial w_2}, \dots, \frac{\partial E}{\partial w_m} \right)$$

To adjust the weight w_{ij} , which connects neuron i to neuron j , gradient descent updates the weight in the negative gradient direction:

$$\Delta w_{ij} = -a \cdot \frac{\partial E}{\partial w_{ij}}$$

$$\Delta w_{ij} = -a \cdot \sum_{j=1}^m (o_{ij} - t_{ij}) \cdot (-x_i) = a \cdot \sum_{j=1}^m (o_{ij} - t_{ij}) \cdot x_i$$

where a represents the predefined learning rate. The adjusted weight is then computed as:

$$w_{ij}^{\text{new}} = w_{ij}^{\text{old}} + \Delta w_{ij}$$

Because neural networks have many parameters, each update changes the model in many dimensions at once. The learning rate therefore has to be chosen carefully: large steps can destabilize training, while very small steps can make learning inefficient.



Specialized NNs

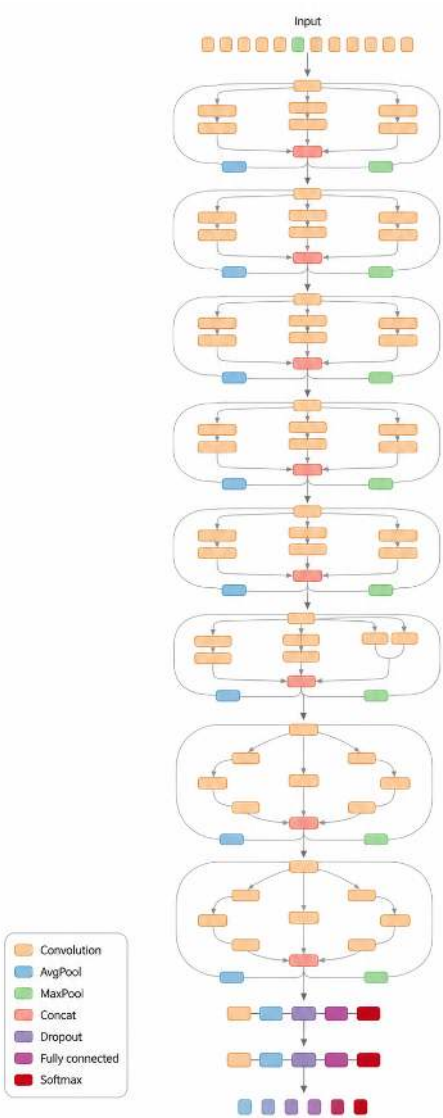


Basic neural networks are often introduced as **fully connected feed-forward networks**. Specialized architectures go beyond this and differ in the following design choices:

Design choice	How architectures differ
Input structure assumed	Different architectures are built for different data structures, such as tabular data (MLPs), spatial data and images (CNNs), sequence data (RNNs or Transformers), relational data (graph neural networks), or ; generative tasks (GANs).
Neuron / unit type	The basic unit may be a dense neuron, recurrent cell, gated LSTM/GRU cell, attention head, or a generator/discriminator module.
Layer operation	Layers may perform matrix multiplication, convolution with moving filters, recurrent state updates, self-attention, or adversarial generator/discriminator training.
Connectivity pattern	Information may flow through all-to-all dense connections, local sliding windows, temporal loops with hidden states, or token-to-token attention.

Focus in this section: We use **CNNs**, **RNNs**, and **Transformers** as key examples because they show three central architectural ideas:

- Convolution through moving filters
- Memory through recurrence, and
- Attention.

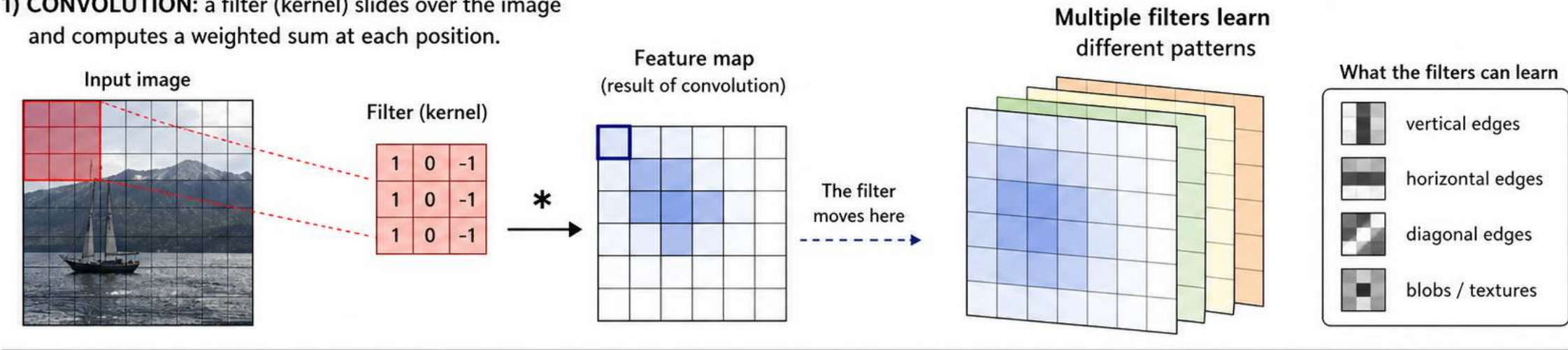


Example architecture: Inception-ResNet-v2

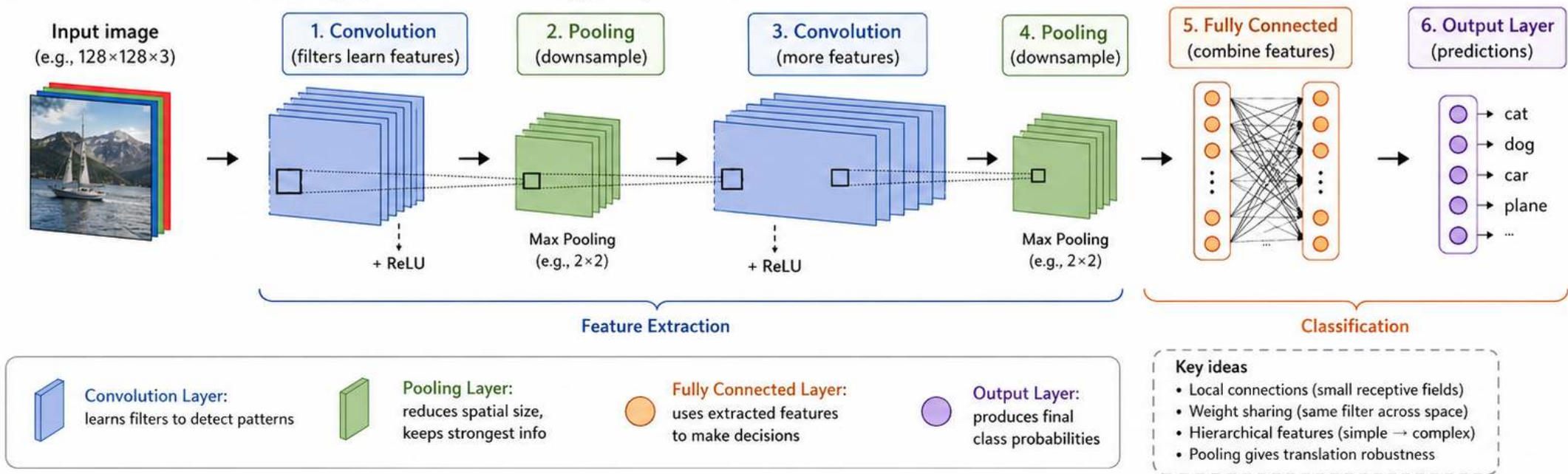
Convolutional neural networks (CNN)



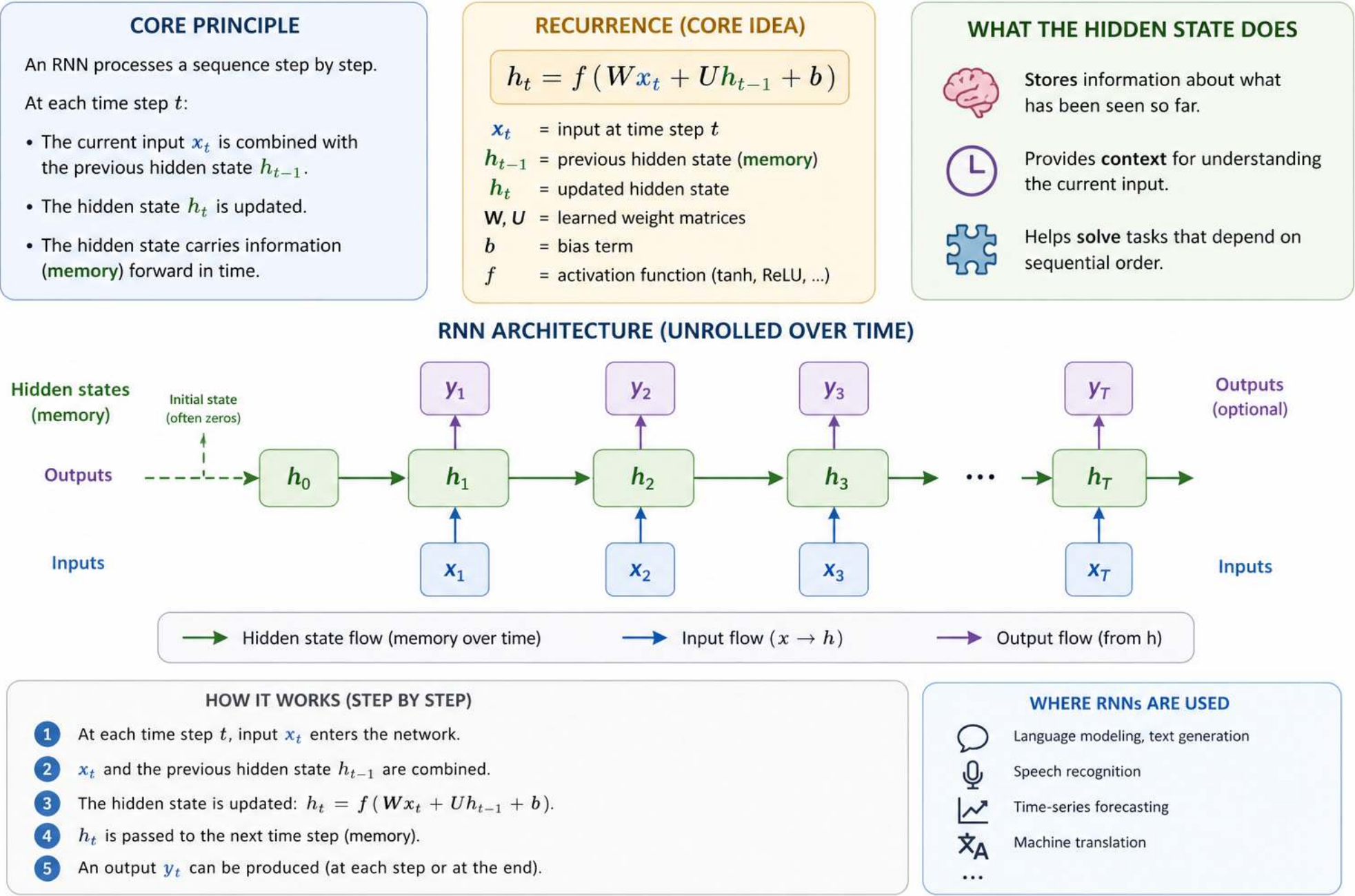
1) **CONVOLUTION:** a filter (kernel) slides over the image and computes a weighted sum at each position.



2) **CNN ARCHITECTURE:** stacking layers to learn increasingly complex features

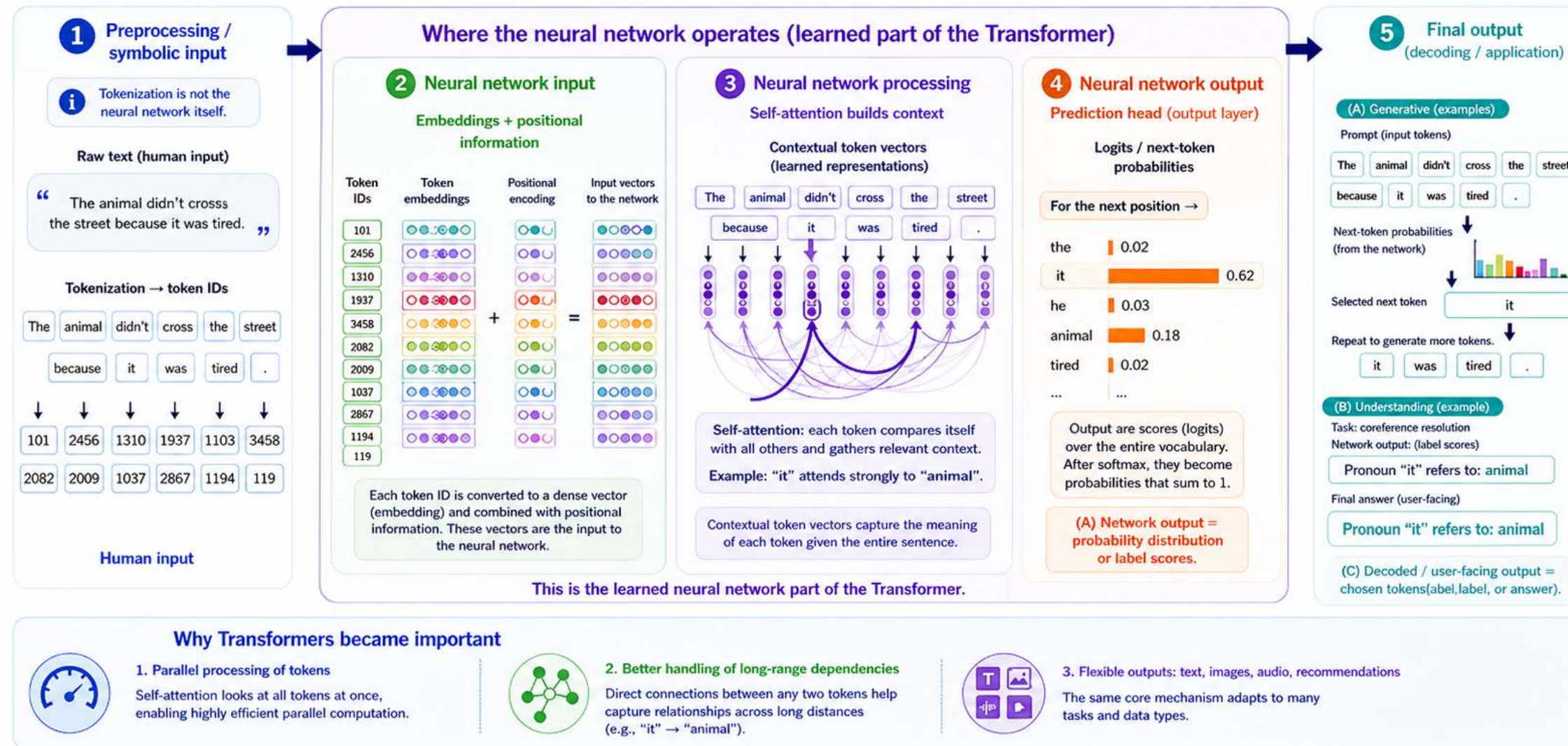


Recurrent neural networks (RNN)



Transformer models: from input to output

Focus: representations, context, and generation



Design and training of neural networks

Modeling: choose the architecture and training setup



💡 **Best-practice logic** Start with choices that are mostly determined by the **data structure** and the **prediction task**. Then tune the parts that require experimentation.

Architecture family

Data structure	Typical model	Inductive bias
 Tabular data	MLP	fully connected
 Images	CNN	local / spatial
 Audio	CNN / RNN / Transformer	local, sequential, attention
 Time series	RNN / Transformer	temporal order
 Text / language	Transformer	global attention

Output layer and loss function

Prediction task	Output layer	Output activation	Loss function
Regression	1 neuron	Linear	MSE
Binary classification	1 neuron	Sigmoid	Binary cross-entropy
Multi-class classification	One neuron per class	Softmax	Categorical cross-entropy

Rule of thumb: data structure determines the architecture; prediction task determines the output layer and loss.

Training setup choices



Besides the selection of the model type, output layer, and loss function (see previous slide), these choices are usually standardized:

- **Adam** is a strong default optimizer
- Hidden layers usually start with **ReLU**

These choices require **more experimentation** to tune complexity and parameters:

1. Network structure

How much can the model represent?

Depth

number of layers

- shallow → simple patterns
- deep → complex abstractions

Width

neurons per layer

Typical starts:

32, 64, 128, 256

2. Learning rate η

How large are the update steps?

$$w := w - \eta \nabla_w J$$

- too high → unstable
- too low → slow

Typical range:

10^{-3} to 10^{-4}

3. Regularization

How do we reduce overfitting?

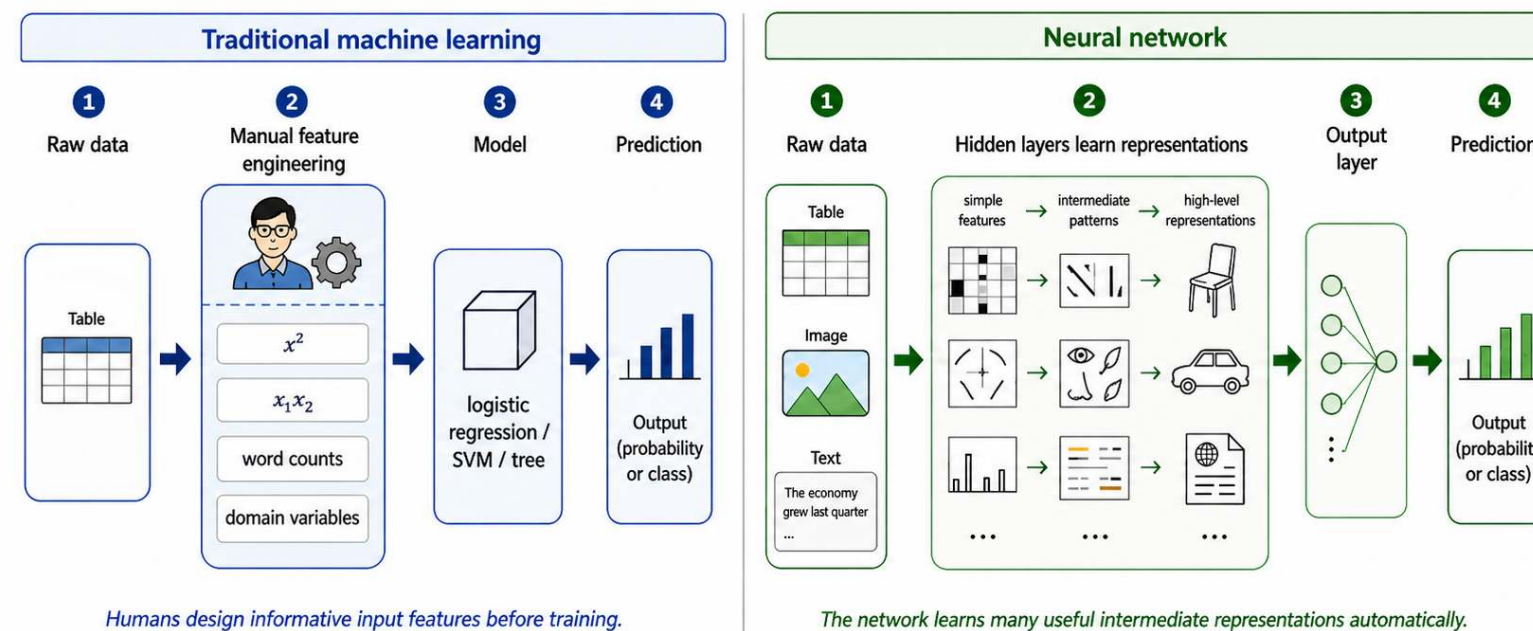
- **Dropout**
randomly disables units
- **Weight decay**
penalizes large weights
- **Early stopping**
stops before overfitting

Note: Neural networks and feature engineering



Traditional machine learning often relies strongly on **manual feature engineering**: humans design useful input variables before training the model. Neural networks can reduce this need because hidden layers learn **intermediate representations** from data. However, feature engineering does not disappear. We still need to decide:

- which data are provided as input
- how inputs are encoded and normalized
- whether domain-specific variables or transformations are added



Neural networks reduce, but do not eliminate, the need for manual feature engineering.

Python code example

The `sklearn` library provides basic feedforward networks, including `MLPClassifier` and `MLPRegressor`. It does not offer CNNs, RNNs, or other advanced neural networks.

```

1 from sklearn.neural_network import MLPClassifier
2 from sklearn.datasets import make_classification
3 from sklearn.model_selection import train_test_split
4
5 X_train, X_test, y_train, y_test = train_test_split(X, y)
6
7 model = MLPClassifier(
8     hidden_layer_sizes=(32, 16), # architecture (depth + width)
9     activation="relu",           # hidden activation
10    solver="adam",               # optimizer
11    learning_rate_init=0.001,    # learning rate
12    alpha=0.0001,               # L2 regularization (weight decay)
13    max_iter=200
14 )
15 model.fit(X_train, y_train)

```

PyTorch (I)



For advanced neural networks and production environments, [pytorch](#) and [tensorflow](#) are suitable libraries.

```
1 import torch
2
3 X = torch.randn(100, 10)
4 y = torch.sum(X, dim=1, keepdim=True) # simple target
5
6 # TODO: specify MLP
7
8 model = MLP(
9     input_dim=10,
10    hidden_layers=[32, 16], # try: [8], [64, 64], [128, 64, 32]
11    output_dim=1 # regression
12 )
13
14 # Training setup
15 criterion = nn.MSELoss() # regression
16 learning_rate = 1e-3 # try: 1e-1, 1e-4
17 optimizer = torch.optim.Adam(model.parameters(), lr=learning_rate)
18
19 # Training loop
20 for epoch in range(100):
21
22     predictions = model(X) # Forward pass
23     loss = criterion(predictions, y)
24
```

PyTorch (II)



```
1 import torch.nn as nn
2
3 # Specify MLP
4 class MLP(nn.Module):
5     def __init__(self, input_dim, hidden_layers, output_dim):
6         super().__init__()
7
8         layers = []
9         prev_dim = input_dim
10
11         # Depth and width
12         for h in hidden_layers:
13             layers.append(nn.Linear(prev_dim, h))
14             layers.append(nn.ReLU())
15             prev_dim = h
16
17         layers.append(nn.Linear(prev_dim, output_dim))
18
19         self.model = nn.Sequential(*layers)
20
21     def forward(self, x):
22         return self.model(x)
23
24 #
```

See [torch.nn.Module](#) for inherited methods.

Summary



- Neural networks transform inputs into outputs through **weighted connections**, **biases**, and **activation functions**.
A single perceptron can only learn **linear decision boundaries**; multilayer networks with nonlinear activations can model more complex patterns.
- Training consists of a **forward pass**, loss calculation, **backpropagation**, and parameter updates via an optimizer such as gradient descent or Adam.
- Different neural network architectures encode different assumptions about data:
MLPs for tabular data, **CNNs** for spatial data, **RNNs** for sequences, and **Transformers** for attention-based language tasks.
- Designing neural networks means choosing the architecture, output layer, loss function, learning rate, optimizer, and regularization strategy, then iteratively evaluating and adjusting these choices.

Survey: Session 9



<https://forms.gle/1vsCsqc3SzWfSX1f6>

Note: Responses may be analyzed and published in anonymized form.

Please complete the survey before you leave today — thank you 🙏

References

Zhang, A., Lipton, Z. C., Li, M., & Smola, A. J. (2023). *Dive into deep learning*. Cambridge University Press.

